

Programación II

Unidad 3: Estructuras de datos.

"Si ha elegido las estructuras de datos correctas y las cosas están bien organizadas, los algoritmos casi siempre serán evidentes."

~ Rob Pike

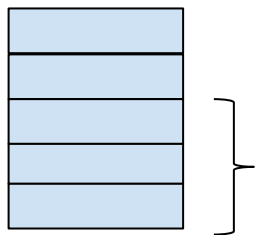
Estructuras de datos

En esta unidad vamos a estudiar el manejo de estructuras de datos dinámicas.

¿Por qué dinámicas? Porque a diferencia de los vectores y matrices cuyo tamaño es fijo y no varía durante la ejecución del programa, las estructuras dinámicas pueden crecer o reducirse a medida que se ejecuta el programa.

Antes de empezar a analizar cada una de las estructuras, vamos a aclarar algunos conceptos sobre la memoria.

Cuando un proceso es ejecutado, el mismo se divide en 2 grandes segmentos: código y datos. A su vez, el segmento de datos se subdivide en: datos variables, stack y heap.



- Código: contiene el código ejecutable del programa
- Datos: contiene variables locales globales y estáticas.
- Stack: utilizado para “apilar” las llamadas a funciones y sus parámetros.
- Heap: contiene variables cuya reserva de memoria es dinámica. Este segmento puede crecer o disminuir su tamaño.

El *stack* (o pila) es la porción de memoria que el sistema operativo utiliza para almacenar las llamadas a función y crear las variables locales de la función y sus parámetros. Es una porción de memoria que el sistema operativo maneja con un algoritmo tipo LIFO (último entrado, primero en salir) por lo que decimos que “apila” las llamadas a funciones. Cuando invocamos a una función, la misma (junto con sus parámetros y variables locales) se crean en el stack. Cuando la función retorna, todas las variables creadas en el stack son destruidas liberando el espacio utilizado.

Las estructuras dinámicas estarán alojadas en el segmento denominado *heap*. Este segmento es parte de la memoria RAM y pertenece al espacio de memoria asignado al proceso que se está ejecutando. A diferencia del *stack* no hay ningún algoritmo para almacenar los datos y la responsabilidad por la reserva y liberación de memoria se delega al programador.

La descripción anterior, refiere al comportamiento de una aplicación desarrollada utilizando lenguajes como C o C++ donde es posible utilizar funciones (brindadas por el lenguaje) que permiten manipular la memoria del proceso a bajo nivel.

En lenguajes de alto nivel como Java, Python, Javascript esto no es necesario ya que existen otros mecanismos automáticos que gestionan el uso de la memoria del proceso.

Las estructuras dinámicas que estudiaremos son: pilas, colas y listas.

Tipo de dato

Si bien, estas estructuras pueden almacenar cualquier tipo de dato, generalmente crearemos una clase a la que llamaremos *Nodo*.

La forma más genérica de definir esta clase es la siguiente:

```
class MyNode<T> {
    private _value: T;
    private _next: MyNode<T>;

    constructor(value: T) {
        this._value = value;
        this._next = undefined as unknown as MyNode<T>;
    }

    public get value(): T {
        return this._value;
    }

    public get next(): MyNode<T> {
        return this._next;
    }

    public set next(n: MyNode<T>) {
        this._next = n;
    }
}
```

Ejemplo 43.1

Nótese el uso del tipo de dato genérico (generics) que nos permite construir nodos que almacenan valores de tipo number, string u objetos de otro tipo.

Pilas

Son estructuras de datos que almacenan y recuperan sus elementos siguiendo un orden estricto. Este orden es *LIFO*, es decir, último en entrar primero en salir.

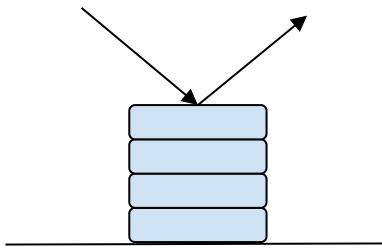
Definición

es una colección de elementos de un mismo tipo de dato a los que sólo se puede acceder (introducir y eliminar) por un único lugar denominado cabeza de pila.

En general, el uso de este tipo de estructura es adecuado cuando necesitamos invertir el orden de un conjunto de elementos o bien conocer el orden en que se fueron sucediendo determinados eventos, como por ejemplo el historial de navegación de páginas web, el historial de acciones de un programa para luego poder “deshacerlas”, etc.

Es importante destacar que la lectura sobre este tipo de estructura de datos es destructiva, es decir, para leer un elemento debemos removerlo de la pila.

Imaginemos una pila de platos o libros, el primer plato que agrego a la pila será el último que pueda extraer de la misma.



En este ejemplo, el primer libro agregado a la pila es el “Libro 1”, el segundo es el “Libro 2”, y así sucesivamente hasta el “Libro N”.

Cuando necesitemos leer los elementos de la pila, lo haremos en orden inverso, primero vamos a leer el “Libro N” y por último el “Libro 1”.

Definición de la clase pila

```
class Stack<T> {
  private head: MyNode<T>;

  public constructor() {
    this.head = undefined as unknown as MyNode<T>;
  }

  public push(value: T): void {
    const newNode = new MyNode(value);
    newNode.next = this.head;
    this.head = newNode;
  }

  public pop(): T | undefined {
    if (this.isEmpty()){
      return undefined;
    }
  }
}
```

```
}  
  
const value = this.head.value;  
this.head = this.head.next;  
return value;  
}  
  
public isEmpty(): boolean {  
    return this.head === undefined;  
}  
}
```

Ejemplo 43.2

Operaciones sobre una pila

create (crear):

Para crear una pila no hay que realizar ninguna acción más que inicializar la cabecera de la pila, por lo que sólo debemos invocar al constructor de la clase.

isEmpty (está vacía)

Esta operación nos permite saber si la pila está vacía o no. Al igual que la operación de creación, es muy simple ya que sólo debemos verificar si la cabecera de la pila es *undefined*.

push (insertar)

La operación push nos permite insertar un elemento nuevo a la pila. Recordemos que la estructura de datos pila siempre inserta elementos en el tope de la pila, es decir en la cabecera.

pop (eliminar)

La operación pop nos permite eliminar un elemento de la pila. Recordemos que la estructura de datos pila siempre elimina elementos del tope de la pila, es decir de la cabecera.

Cabe aclarar que las operaciones de lectura sobre una pila son destructivas, en el sentido que para leer los elementos de la pila, debemos removerlos de la misma utilizando la función `pop`.

Tener en cuenta que antes de invocar a la función `pop` debemos verificar que la pila tenga elementos ya que de lo contrario producirá un error.

Ejemplo

```
// Crear una pila que almacena números
const p = new Stack<number>();

// Verificar estado de la pila. Salida esperada: true
console.log(p.isEmpty());

// Agregar elementos a la pila
p.push(1);
p.push(2);
p.push(4);

// Imprimir el contenido de la pila. Salida esperada 4, 2, 1
while(!p.isEmpty()) {
    console.log(p.pop());
}

// Verificar estado de la pila. Salida esperada: true
console.log(p.isEmpty());
```

Ejemplo 43.3

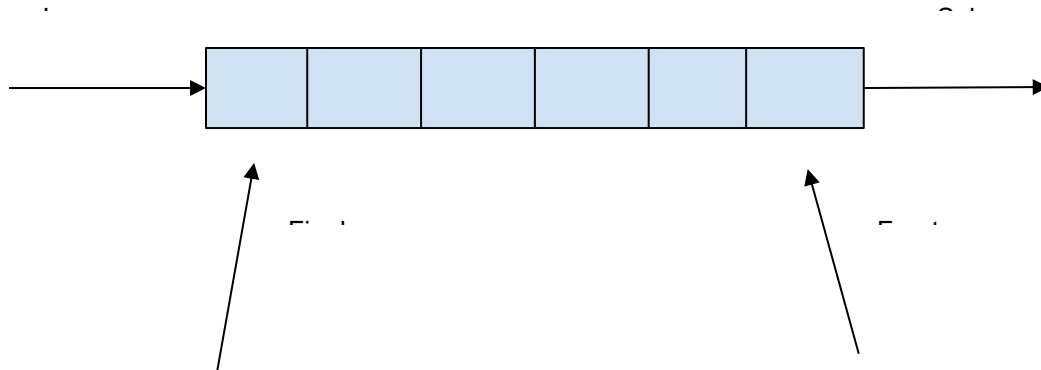
Colas

Al igual que las pilas, son estructuras de datos que almacenan y recuperan sus elementos siguiendo un orden estricto, pero a diferencia de éstas, el orden es *FIFO*, es decir, primero en entrar primero en salir.

En cierto modo, las *colas* son parecidas a las *pilas* en el sentido de que son una colección de elementos que se insertan por un extremo (parte final), pero la diferencia con estas últimas es que la extracción de dichos elementos no se realiza por el mismo extremo por el cual se insertaron sino que se realiza por el frente (parte inicial o cabecera).

Al igual que las pilas, la operación de lectura de los elementos es una operación destructiva ya que el elemento es removido de la cola.

Imaginemos una fila de personas que esperan para ser atendidas por el cajero de un banco o una lista de trabajos que esperan su turno para hacer uso del procesador.



Definición de la clase cola

```
class Queue<T> {
  private first: MyNode<T>;
  private last: MyNode<T>;

  public constructor() {
    this.first = undefined as unknown as MyNode<T>;
    this.last = undefined as unknown as MyNode<T>;
  }

  public enqueue(value: T): void {
    const newNode = new MyNode(value);

    if (!this.first) {
      this.first = newNode;
    }
    else {
      this.last.next = newNode
    }

    this.last = newNode
  }

  public dequeue(): T | undefined {
    if (this.isEmpty()){
      return undefined;
    }

    const value = this.first.value;
    this.first = this.first.next;
    if (this.first === undefined) {
      this.last = undefined as unknown as MyNode<T>;
    }
    return value;
  }

  public isEmpty(): boolean {
```

```
    return this.first === undefined && this.last === undefined;
  }
}
```

Ejemplo 43.4

Operaciones sobre una cola

create (crear):

Para crear una pila no hay que realizar ninguna acción más que inicializar la los punteros de inicio y fin de la cola, por lo que sólo debemos invocar al constructor de la clase.

Para manejar una cola, utilizaremos una estructura que contenga ambos punteros, cabecera y final.

enqueue (agregar)

La operación enqueue nos permite agregar un elemento nuevo a la cola. Recordemos que la estructura de datos cola siempre inserta elementos en el final de la cola.

dequeue (eliminar)

La operación dequeue nos permite eliminar un elemento de la cola. Recordemos que la estructura de datos cola siempre elimina elementos del frente de la cola.

Cabe aclarar que las operaciones de lectura sobre una cola son destructivas, en el sentido que para leer los elementos de la cola, debemos removerlos de la misma utilizando la función `remove`.

Tener en cuenta que antes de invocar a la función `remove` debemos verificar que la cola tenga elementos ya que de lo contrario producirá un error.

isEmpty (está vacía)

Esta operación nos permite saber si la cola está vacía o no. Al igual que la operación de creación, es muy simple ya que sólo debemos verificar si los punteros de la cola son *undefined*.

Ejemplo

```
// Crear una cola que almacena números
const q = new Queue<number>();

// Verificar que esté vacía. Salida esperada: true
console.log(q.isEmpty());

// Encolar elementos
q.enqueue(1);
q.enqueue(2);
q.enqueue(4);

// Imprimir el contenido de la cola. Salida esperada 1, 2, 4
while(!q.isEmpty()) {
    console.log(q.dequeue());
}

// Verificar que esté vacía. Salida esperada: true
console.log(q.isEmpty());
```

Ejemplo 43.5

Listas

Una lista enlazada es una estructura de datos que maneja una colección de elementos dispuestos uno detrás de otro en la que cada elemento se conecta al siguiente mediante un puntero.

A diferencia de las *pilas* y *colas* no posee una política definida para el manejo de sus elementos, motivo por el cual una lista puede ser recorrida en cualquier orden, se puede insertar elementos en cualquier parte de la misma y las operaciones de lectura no son destructivas.

Al igual que con las otras estructuras de datos con las que estuvimos trabajando, para trabajar con listas debemos definir la estructura del nodo, que consta de datos y como mínimo un puntero al siguiente nodo.

Antes de comenzar a analizar las operaciones, hay que destacar que a diferencia de las estructuras vistas anteriormente, las operaciones que insertan un nodo en la lista también lo devuelven. Esta forma de trabajar nos va a permitir desarrollar estructuras más complejas como *listas de listas*.

Definición de la clase lista

```
class List<T> {
  private head: MyNode<T>;

  constructor() {
    this.head = undefined as unknown as MyNode<T>;
  }

  public push(value: T): MyNode<T> {
    const node = new MyNode(value);

    let headAux = this.head;
    while(headAux && headAux.next) {
      headAux = headAux.next;
    }
    if (!headAux) {
      this.head = node
    }
    else {
      headAux.next = node;
    }

    return node;
  }

  public pop(): T {
    let value = undefined;
    let headAux = this.head;
    let previous: MyNode<T> = undefined as unknown as MyNode<T>;
    while(headAux && headAux.next) {
      previous = headAux;
      headAux = headAux.next;
    }

    if (!previous) {
      this.head = undefined as unknown as MyNode<T>;
    }
    else {
      previous.next = undefined as unknown as MyNode<T>;
    }
    value = headAux.value;
    return value;
  }

  public insertFirst(value: T): MyNode<T> {
    const node = new MyNode(value);
    node.next = this.head;
    this.head = node;
    return node;
  }
}
```

```
public removeFirst(): T {
    let value = undefined as unknown as T;
    if (this.head) {
        value = this.head.value;
        this.head = this.head.next;
    }

    return value;
}

public insertOrdered(value: T): MyNode<T> {
    const node = new MyNode(value);
    let headAux = this.head;
    let previous: MyNode<T> = undefined as unknown as MyNode<T>;
    while(headAux && headAux.value < value) {
        previous = headAux;
        headAux = headAux.next;
    }

    if (!previous) {
        this.head = node;
    }
    else {
        previous.next = node;
    }
    node.next = headAux;

    return node;
}

insertUnique(value: T): MyNode<T> {
    const node = this.search(value);
    if (!node) {
        this.insertOrdered(value);
    }
    return node;
}

public delete(value: T): T {
    let headAux = this.head;
    let previous: MyNode<T> = undefined as unknown as MyNode<T>;
    while(headAux && headAux.value !== value) {
        previous = headAux;
        headAux = headAux.next;
    }

    if (!previous) {
        this.head = headAux.next;
    }
    else {
        previous.next = headAux.next;
    }

    return headAux.value;
}
```

```

    }

    public search(value: T): MyNode<T> {
        let headAux = this.head;
        while(headAux && headAux.value !== value) {
            headAux = headAux.next;
        }
        return headAux;
    }

    public sort(): void {
        let value: T;
        let listAux = new List<T>();
        while(this.head) {
            value = this.removeFirst();
            listAux.insertOrdered(value);
        }
        this.head = listAux.head;
        listAux.clear();
    }

    public clear(): void {
        while(this.head){
            this.removeFirst();
        }
    }
}

```

Ejemplo 43.6

Operaciones sobre una lista

Create (crear):

Para crear una lista no hay que realizar ninguna acción más que inicializar la cabecera de la lista, por lo que sólo debemos invocar al constructor de la clase.

insertFirst (inserta adelante)

Esta operación agrega un nodo delante del primer nodo.

push (inserta al final)

Esta operación agrega un nodo al final de la lista.

En esta función debemos destacar la utilización del puntero auxiliar *headAux*. Este puntero es utilizado para no perder la referencia al primer nodo de la lista.



insertOrdered (inserta ordenado)

Esta operación permite insertar un nodo conservando el orden, en otras palabras, esta función permite insertar un nodo respetando el orden de los datos.

search (buscar un nodo)

Esta función permite buscar y devolver el nodo que contenga el valor pasado por parámetro.

removeFirst (elimina el primer nodo)

Esta operación permite eliminar el primer nodo de la lista (similar al manejo de pilas)

pop (elimina el último nodo)

Esta operación permite eliminar el último nodo de la lista

delete (elimina un nodo particular)

Esta función permite eliminar el nodo que contiene el valor pasado por parámetro.

clear (vaciar la lista)

Esta operación permite vaciar la lista

SortList (ordenar una lista)

Esta función permite ordenar una lista. Para llevar a cabo esta tarea vamos a reutilizar las funciones *removeFirst* e *insertOrdered*.

insertUnique (inserta sin repetición)

Esta función permite insertar un nodo siempre y cuando el mismo no exista previamente en la lista. En caso de no existir lo inserta respetando el orden de la lista. Devuelve el puntero al nodo encontrado o insertado.



Bibliografía utilizada:

- Luis Joyanes Aguilar, Ignacio Zahonero Martínez. Programación en C. Segunda Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA. S.A.U., 2005. ISBN-84: 481-9844-1.
- Brian W. Kernighan, Rob Pike. La práctica de la programación. Pearson Educación. Méjico (2000)
- Debugging with CodeBlocks:
[http://wiki.codeblocks.org/index.php?title=Debugging with Code::Blocks](http://wiki.codeblocks.org/index.php?title=Debugging_with_Code::Blocks)
- Bibliotecas.
 - [https://es.wikipedia.org/wiki/Biblioteca_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Biblioteca_(inform%C3%A1tica))
- Enlace dinámico
 - https://es.wikipedia.org/wiki/Enlace_din%C3%A1mico
- Enlace estático
 - https://es.wikipedia.org/wiki/Enlace_est%C3%A1tico